

Azure Terraform Labs

Day 1 Detailed Notes

Basic Azure Infrastructure • Remote State • State Locking • Drift • Brownfield Import

Purpose: Document the full first Azure Terraform session, including the commands, why each step was needed, the questions that came up, and the final mental model.

1. What I Built and Learned

This session started with connecting a local WSL/VS Code environment to a temporary Azure sandbox, then progressed from a small Terraform deployment to real Terraform operations such as remote state, state locking, drift remediation, and importing an existing resource.

- Connected VS Code / WSL to Azure with Azure CLI.
- Configured Terraform and the AzureRM provider.
- Used an existing Azure Resource Group through a data source because the sandbox blocked Resource Group creation.
- Built a VNet, subnet, NSG, NSG association, Public IP, NIC, and Linux VM.
- Created an SSH key pair and connected to the VM.
- Started with local Terraform state.
- Created an Azure Storage Account and Blob Container for remote state.
- Migrated the local state into the AzureRM backend.
- Observed Terraform acquiring and releasing a remote state lock.
- Created configuration drift manually in the Azure Portal, detected it with terraform plan, and repaired it with terraform apply.
- Created an NSG manually in Azure and imported it into Terraform state as a brownfield resource.
- Finished with terraform plan showing No changes.

Architecture

```
Internet
|
v
Public IP
|
v
Network Interface
|
+---- Private IP: 10.10.1.x
|
v
Linux VM
|
v
Subnet: 10.10.1.0/24
|
v
VNet: 10.10.0.0/16

NSG
|
+---- Inbound TCP/22 SSH
|
+---- Associated with subnet
```

Terraform state after Lab 2

```
Azure Storage Account
|
```

```
      v
Blob Container: tfstate
      |
      v
Lab01.terraform.tfstate
```

2. Azure CLI Setup

Install Azure CLI

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

Command breakdown:

- `curl` - downloads content from a URL.
- `-s` - silent mode, reducing extra output.
- `-L` - follows redirects.
- `|` - pipes the downloaded installer into the next command.
- `sudo bash` - runs the installer with administrator privileges.

Then verify the installation:

```
az --version
```

The installed Azure CLI version was 2.90.0.

Authenticate to Azure

```
az login
```

This authenticated the local Azure CLI session against the Pluralsight sandbox account. Terraform later reused this Azure CLI authentication for local development.

```
Local WSL / VS Code
|
v
Azure CLI
|
v
Azure login
|
v
Pluralsight Azure subscription
```

Verify the active subscription:

```
az account show --output table
```

This is important in real environments because a user may have access to development, testing, and production subscriptions. Always confirm the active subscription before deploying infrastructure.

Question: Why no Service Principal yet?

For local interactive work, `az login` is simple and appropriate. A Service Principal or workload identity is used when software or automation needs to authenticate without a human.

```
Local development:
Human -> az login -> Azure CLI -> Terraform -> Azure

CI/CD:
GitHub Actions -> Workload Identity / Service Principal -> Terraform -> Azure
```

A Service Principal is an application identity. In an infrastructure CI/CD pipeline, GitHub Actions or another runner can use an Azure identity with RBAC permissions so Terraform can run plan/apply without a person entering credentials.

3. Terraform Project Files

```
azure-lab-01/  
├─ main.tf  
├─ providers.tf  
├─ variables.tf  
├─ terraform.tfvars  
├─ version.tf  
├─ .terraform.lock.hcl  
├─ .gitignore  
└─ README.md
```

Terraform reads all .tf files in the same directory as one configuration. File names are for organization; Terraform does not require resources to live in a specific file.

version.tf

```
terraform {  
  required_version = ">= 1.5.0"  
  
  required_providers {  
    azurerm = {  
      source  = "hashicorp/azurerm"  
      version = "~> 4.0"  
    }  
  }  
}
```

This declares the Terraform version requirement and the AzureRM provider dependency. It does not create Azure resources.

providers.tf

```
provider "azurerm" {  
  features {}  
}
```

This configures the AzureRM provider. Because Azure CLI was already authenticated, Terraform could use that local authentication context.

variables.tf and terraform.tfvars

```
variable "location" {  
  description = "Azure region for resources"  
  type        = string  
}  
  
variable "resource_group_name" {  
  description = "Name of the resource group"  
  type        = string  
}  
  
variable "vnet_name" {
```

```

description = "Name of the virtual network"
type        = string
}

variable "subnet_name" {
description = "Name of the subnet"
type        = string
}

```

The variables file defines the inputs; terraform.tfvars supplies the actual values.

```

location          = "West US"
resource_group_name = "rg-tf-lab01"
vnet_name         = "vnet-lab01"
subnet_name       = "subnet-lab01"

```

This separation keeps code reusable. A production project could use the same resource logic with different values for dev, test, and prod.

4. Sandbox Permission Error and Data Sources

The first attempt tried to create a new Resource Group, but Azure returned a 403 AuthorizationFailed error. The Pluralsight sandbox restricted the account to the pre-created Resource Group.

```

resource "azurerm_resource_group" "lab" {
name      = var.resource_group_name
location = var.location
}

```

Instead, the existing Resource Group was referenced with a data source:

```

data "azurerm_resource_group" "lab" {
name = "1-b339435d-playground-sandbox"
}

```

Resource vs data: resource means "create/manage this object"; data means "read information about an already-existing object."

5. Build the Azure Network

Virtual Network

```

resource "azurerm_virtual_network" "lab_vnet" {
name           = var.vnet_name
address_space = ["10.10.0.0/16"]
location      = var.location
resource_group_name = data.azurerm_resource_group.lab.name
}

```

The VNet address space was 10.10.0.0/16.

Subnet

```
resource "azurerm_subnet" "lab_subnet" {
  name                = var.subnet_name
  resource_group_name = data.azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.lab_vnet.name
  address_prefixes    = ["10.10.1.0/24"]
}
```

The line `virtual_network_name = azurerm_virtual_network.lab_vnet.name` is more than a value lookup. It also creates an implicit dependency, so Terraform knows the VNet must exist before the subnet can be created.

Network Security Group

```
resource "azurerm_network_security_group" "lab_nsg" {
  name                = "nsg-lab01"
  location            = var.location
  resource_group_name = data.azurerm_resource_group.lab.name

  security_rule {
    name                = "Allow-SSH"
    priority            = 100
    direction          = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "22"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}
```

For this short sandbox lab, SSH was allowed from any source. In a real environment this should normally be restricted to a known source range, a VPN, Bastion, or another controlled access method.

Associate the NSG with the subnet

```
resource "azurerm_subnet_network_security_group_association" "lab_nsg_assoc" {
  subnet_id                = azurerm_subnet.lab_subnet.id
  network_security_group_id = azurerm_network_security_group.lab_nsg.id
}
```

Creating an NSG alone does not protect a resource. It must be associated with a subnet or NIC. In this lab it was attached to the subnet.

6. Public IP, NIC, and Linux VM

Public IP

```
resource "azurerm_public_ip" "lab_public_ip" {
  name                = "pip-lab01"
  location            = var.location
  resource_group_name = data.azurerm_resource_group.lab.name
  allocation_method   = "Static"
}
```

```
    sku = "Standard"
  }
```

Azure exposes the Public IP as its own resource. It is associated with the NIC rather than directly with the VM.

```
Internet -> Public IP -> NIC -> VM
```

Network Interface

```
resource "azurerm_network_interface" "lab_nic" {
  name            = "nic-lab01"
  location        = var.location
  resource_group_name = data.azurerm_resource_group.lab.name

  ip_configuration {
    name                = "internal"
    subnet_id           = azurerm_subnet.lab_subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = azurerm_public_ip.lab_public_ip.id
  }
}
```

The NIC joins the subnet, receives a dynamic private IP from Azure, and carries the Public IP association.

Linux VM

```
resource "azurerm_linux_virtual_machine" "lab_vm" {
  name            = "vm-lab01"
  resource_group_name = data.azurerm_resource_group.lab.name
  location        = var.location
  size            = "Standard_B1s"
  admin_username  = "azureuser"

  network_interface_ids = [
    azurerm_network_interface.lab_nic.id
  ]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("~/ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts-gen2"
    version   = "latest"
  }
}
```

Create an SSH key pair

```
ssh-keygen -t rsa -b 4096
```

This created `~/ssh/id_rsa` (private key) and `~/ssh/id_rsa.pub` (public key). Terraform placed only the public key on the Azure VM. The private key stayed on the local machine.

7. Terraform Workflow

1. `terraform init` - initialize the working directory and provider plugins.
2. `terraform fmt` - format HCL consistently.
3. `terraform validate` - validate configuration syntax and references.
4. `terraform plan` - preview proposed changes.
5. Review the plan carefully.
6. `terraform apply` - make the approved changes in Azure.

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

After apply, the infrastructure was created successfully.

Check the Public IP from Terraform state

```
terraform state show azurerm_public_ip.lab_public_ip
```

The VM Public IP was 20.253.221.118 during the temporary sandbox session.

SSH to the VM

```
ssh -i ~/.ssh/id_rsa azureuser@20.253.221.118
```

The VM reported a private IPv4 address of 10.10.1.4 on `eth0`, confirming that the NIC received an address from the 10.10.1.0/24 subnet.

8. Lab 2 - Move State from Local Disk to Azure Blob

The first deployment used the default local backend, so Terraform kept its state in `terraform.tfstate` on the local WSL filesystem.

```
Before:
azure-lab-01/
└─ terraform.tfstate
```

The goal of Lab 2 was to keep the same infrastructure but move Terraform's authoritative state into Azure Blob Storage.

Create the Storage Account

```
resource "azurerm_storage_account" "tfstate" {
  name                        = "sytf1ab01state12345"
  resource_group_name        = data.azure_resource_group.lab.name
  location                   = var.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"
```

```
tags = {
  environment = "lab"
}
```

A more advanced `network_rules` example was found in the documentation, but it was intentionally removed because it was not required for this lab. This reinforced an important rule: documentation examples are references, not instructions to copy every argument.

Create the Blob Container

```
resource "azurerm_storage_container" "tfstate" {
  name                       = "tfstate"
  storage_account_id        = azurerm_storage_account.tfstate.id
  container_access_type     = "private"
}
```

Configure the AzureRM backend

```
terraform {
  backend "azurerm" {
    resource_group_name = "1-b339435d-playground-sandbox"
    storage_account_name = "sytf1ab01state12345"
    container_name       = "tfstate"
    key                  = "Lab01.terraform.tfstate"
  }
}
```

Question: Why must Terraform initialize the backend first?

Terraform needs to know where the state is before it can determine what it already manages. Backend initialization therefore happens before normal resources and data sources are evaluated.

```
terraform init
  |
  v
Initialize backend
  |
  v
Locate state
  |
  v
Load state
  |
  v
Evaluate configuration and resources
```

That is why the backend cannot simply use `data.azurerm_resource_group.lab.name` as if it were a normal resource argument: Terraform must find the backend before it can evaluate that data source.

Question: What is the backend key?

The container can hold many blobs. The key identifies which blob is the state file for this particular Terraform configuration.

```
Storage Account
└─ tfstate container
   ├── Lab01.terraform.tfstate
   ├── Lab02.terraform.tfstate
   └─ networking.tfstate
```

Migrate existing state

```
terraform init -migrate-state
```

Terraform detected the previous local state and asked whether it should copy it to the new AzureRM backend. Answering yes migrated the state without moving or recreating the Azure infrastructure.

```
Local terraform.tfstate
    |
    v
Azure Storage Account
    |
    v
tfstate container
    |
    v
Lab01.terraform.tfstate
```

Verify the remote state

```
terraform plan
```

The plan returned:

```
No changes. Your infrastructure matches the configuration.
```

That confirmed Terraform was reading the migrated remote state and still recognized all previously created resources.

State locking

During remote operations Terraform displayed:

```
Acquiring state lock...
...
Releasing state lock...
```

State locking prevents two Terraform processes from writing to the same state simultaneously, which is especially important for team workflows and CI/CD.

9. Lab 3A - Create and Repair Drift

A tag was manually added to the Terraform-managed VNet in the Azure Portal:

```
owner = portal-drift-test
```

Because that tag was not present in the Terraform configuration, the real Azure object no longer matched the desired configuration.

Drift: A managed resource has been changed outside Terraform, so the actual infrastructure

differs from what the Terraform configuration declares.

Detect the drift

```
terraform plan
```

Terraform proposed an in-place update:

```
~ update in-place
tags = {
  - "owner" = "portal-drift-test" -> null
}

Plan: 0 to add, 1 to change, 0 to destroy.
```

Terraform had refreshed the real Azure resource and detected that the extra tag existed outside the declared configuration.

Repair the drift

```
terraform apply
```

Terraform removed the manually added tag and returned Azure to the declared configuration.

```
Portal manual change
  |
  v
Drift
  |
  v
terraform plan
  |
  v
Difference detected
  |
  v
terraform apply
  |
  v
Desired configuration restored
```

10. Lab 3B - Brownfield Import

A new NSG named nsg-import-lab was created manually in the Azure Portal. Terraform had never created or tracked this NSG.

Brownfield: Infrastructure already exists before Terraform is introduced.

Write a matching Terraform resource address

```
resource "azurerm_network_security_group" "imported_nsg" {
  name       = "nsg-import-lab"
  location   = var.location
}
```

```
resource_group_name = data.azurerm_resource_group.lab.name
}
```

Classic terraform import needs a Terraform resource address to import into. The configuration block defines that address.

Get the Azure Resource ID

```
az network nsg show --resource-group 1-b339435d-playground-sandbox --name nsg-import-lab
--query id --output tsv
```

The Azure Resource ID had the structure:

```
/subscriptions/<subscription-id>
/resourceGroups/<resource-group>
/providers/Microsoft.Network
/networkSecurityGroups/nsg-import-lab
```

The Resource ID uniquely identifies the exact Azure object, even if resources elsewhere have the same friendly name.

Import the existing NSG

```
terraform import azurerm_network_security_group.imported_nsg "/subscriptions/2213e8b1-dbc7-4d54-8aff-b5e315df5e5b/resourceGroups/1-b339435d-playground-sandbox/providers/Microsoft.Network/networkSecurityGroups/nsg-import-lab"
```

The left side is the Terraform resource address. The right side is the exact Azure Resource ID.

```
Terraform resource address
azurerm_network_security_group.imported_nsg
      |
      v
    terraform import
      |
      v
Azure Resource ID for nsg-import-lab
```

Terraform returned Import successful! and added the resource mapping to the remote state.

Verify the import

```
terraform plan
```

The final result was:

```
No changes. Your infrastructure matches the configuration.
```

This meant the Terraform configuration, remote state, and actual Azure NSG all matched.

11. Core Mental Model

```
Terraform configuration
      |
      v
Terraform state
      |
      v
Actual Azure infrastructure
```

The configuration represents the desired state. The state records Terraform's mapping to real resources. Azure contains the actual infrastructure. Terraform plans compare the declared configuration, existing state, and refreshed real infrastructure.

Question: Is using AI to write Terraform a problem?

The goal is not to memorize every AzureRM argument. In real work, engineers use documentation, existing modules, code repositories, IDE completion, and increasingly AI assistance. The important skill is being able to design the architecture, understand the generated code, review a plan, troubleshoot errors, and safely modify infrastructure.

A useful standard is:

```
Architecture and reasoning -> understand yourself
Exact provider syntax      -> documentation / references / AI can help
terraform plan             -> you must be able to review and explain it
Errors and drift           -> you must be able to troubleshoot them
```

Future labs will therefore become challenge-first: requirements and partial hints first, then official documentation research, then review and troubleshooting.

12. Git and Terraform Safety

Terraform state and local caches should not be committed to a public repository. A .gitignore was added before git add.

```
# Terraform working directory
.terraform/

# Terraform state
*.tfstate
*.tfstate.*

# Terraform variable files
*.tfvars

# Terraform plan files
*.tfplan

# Terraform crash logs
crash.log
crash.*.log

# Terraform overrides
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# SSH private keys
*.pem
id_rsa
id_ed25519
```

.terraform.lock.hcl should normally be committed because it records selected provider versions and helps keep provider installation reproducible.

13. Command Reference from the Session

Azure CLI

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az --version
az login
az account show --output table
```

SSH

```
ssh-keygen -t rsa -b 4096
ssh -i ~/.ssh/id_rsa azureuser@20.253.221.118
```

Terraform

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
terraform state show azurerm_public_ip.lab_public_ip
terraform init -migrate-state
terraform import ...
```

Azure Resource ID lookup

```
az network nsg show --resource-group 1-b339435d-playground-sandbox --name nsg-import-lab --query id --output tsv
```

14. Key Lessons

7. The AzureRM provider connects Terraform to Azure APIs.
8. Variables separate reusable configuration from environment-specific values.
9. A Terraform data source reads an existing resource instead of creating it.
10. Resource references create implicit dependencies.
11. Azure VMs connect to subnets through NICs.
12. A Public IP is associated with the NIC.
13. An NSG must be associated with a subnet or NIC before it affects traffic.
14. Terraform state is the mapping between configuration and managed real resources.
15. Remote state is better suited to team workflows than a local state file.
16. Azure Blob Storage can be used as an AzureRM remote backend.
17. The backend must be initialized before normal configuration can be evaluated.
18. The backend key identifies the exact state blob inside the container.
19. terraform init -migrate-state can move state without recreating infrastructure.
20. Remote state can be locked during Terraform operations.

21. terraform plan can detect drift caused by Portal changes.
22. terraform apply can restore a Terraform-managed resource to the declared desired state.
23. terraform import can adopt existing brownfield infrastructure into state.
24. After an import, terraform plan is critical to verify that code, state, and reality match.
25. Documentation examples should be understood and adapted instead of copied blindly.

15. Planned Progression

The next labs will become progressively more advanced while keeping the same requirement -> research -> build -> troubleshoot learning style.

```
Large Azure brownfield import
  |
  v
Import blocks / generated configuration
  |
  v
count and for_each
  |
  v
locals and functions
  |
  v
conditional expressions
  |
  v
multiple environments
  |
  v
custom modules
  |
  v
module inputs and outputs
  |
  v
advanced state operations
  |
  v
intentional drift and break/fix labs
  |
  v
Azure networking projects
  |
  v
Terraform CI/CD
  |
  v
GitHub Actions + Azure workload identity
```

Final Result

```
Azure infrastructure deployed with Terraform
  |
  v
Linux VM reachable over SSH
  |
  v
```

State migrated from local disk to Azure Blob

|

v

Remote state locking observed

|

v

Portal drift detected and repaired

|

v

Existing NSG imported into remote state

|

v

Final terraform plan = No changes

This document records a hands-on learning lab in a temporary Azure sandbox. Resource names, IDs, and IP addresses are lab-specific and temporary.